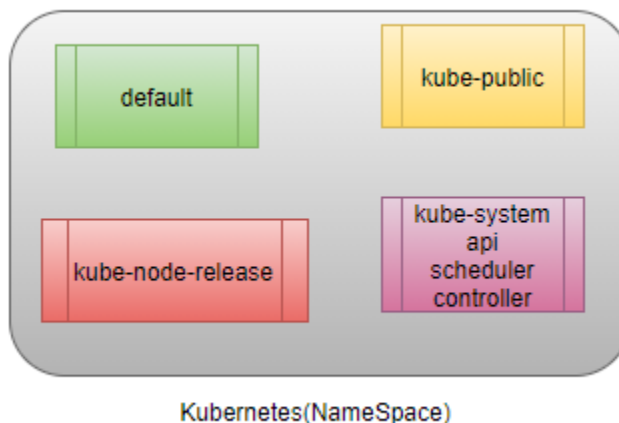
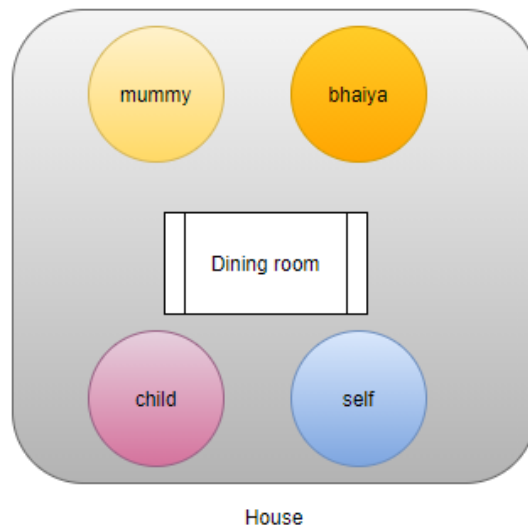


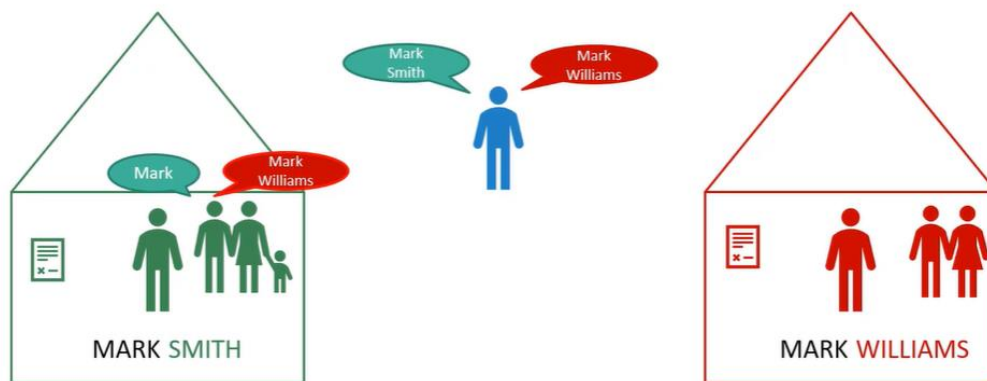
Namespace:

Provides isolation to the resources we create in K8s cluster.

What is a resource? Any thing we create in k8s is called a resource ex. pod.

- Every resource that we create in k8s will always go under namespace.
- When we create/delete/get/describe a resource in k8s, if we don't mention the namespace, it will always go under the "default" namespace.
- Within namespace, we cannot have duplicate names.
- Namespace, allow you to restrict the access to the user who can access what?
- Virtual Cluster inside K8s cluster.





Use Case:

If family member of Mark Willam inside from home wants to call Mark Willam, will identified him from his first name Mark, similarly Mark Sam also Identified by Mark in his home, but the scenario is both are studying in same class and if their teacher wants to call Mark Willam, if they call him Mark then both Mark Willam and Mark Sam will be responds, means to identify them teacher has to call by their full name. Each of the home has their own set of roles which define who can do what? Each of home has their own set of resources which they can consume.

In K8s each resource (pod, controller, service) which we create, created in their own house (default namespace), each namespace has their own sets of resources.

Physical cluster?

`kubectl get nodes`

We have three nodes, and nodes become physical Cluster, we can login on any node can do anything, we can feel it.

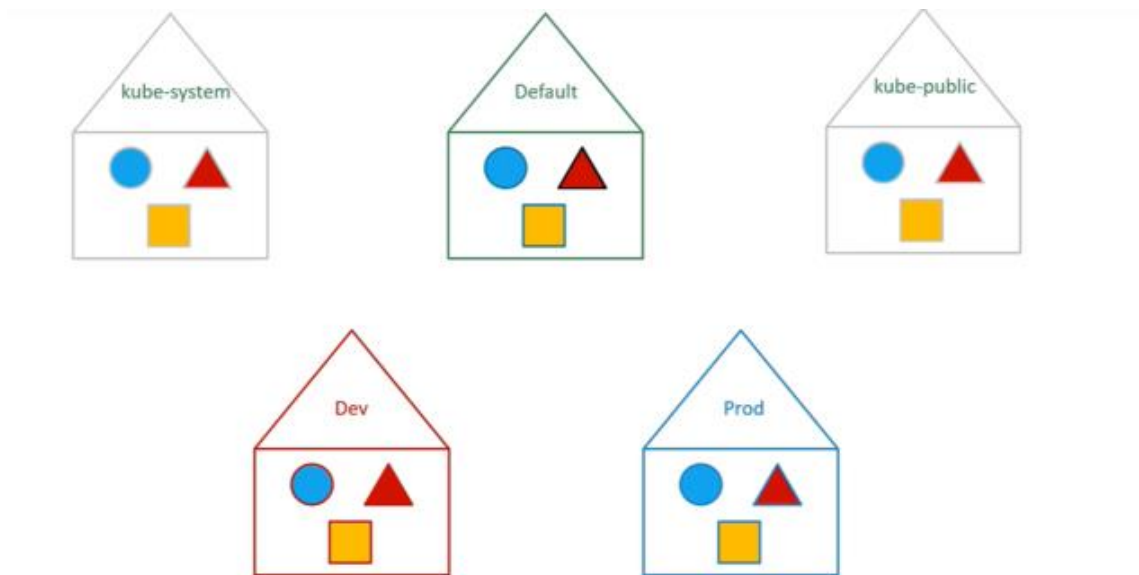
If we want to group set of things, if we want to create virtual space or virtual cluster inside my K8s cluster then we called this virtual space as a namespace.

`kubectl get ns`

```

root@kmaster: ~
root@kmaster:~# kubectl get ns
NAME                STATUS   AGE
default              Active   6d13h
kube-node-lease      Active   6d13h
kube-public          Active   6d13h
kube-system          Active   6d13h

```



kube-system

designed to run only your system services, only admin can put something in kube-system namespace.

kube-public

if we deploy any resource in public name space will be available publicly to resource of other namespace, will be available to all the users in the cluster. Just like /tmp folder. Don't use kube-public namespace.

- publicly accessible data

kube-node-lease

We not be able to use this namespace, used only by api server to check the health check status of node, its internally used by api-server

- heartbeats of the nodes
- each node has associated lease object in namespace
- determine the availability of the node

default

whenever we create a resource without telling the home of the resource then by default resource created inside default name space. **It is created by default by k8s during cluster setup.**

How to create custom namespace?

Imperative approach

kubectl create ns <name of namespace>

kubectl create ns dev

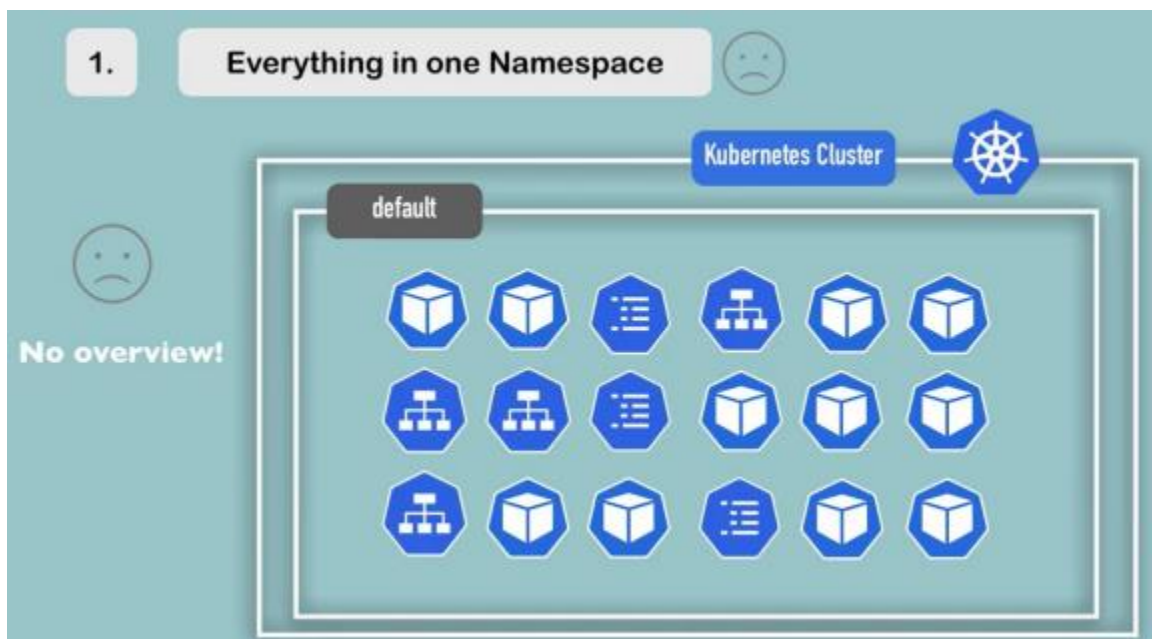
kubectl create ns prod

Declarative approach

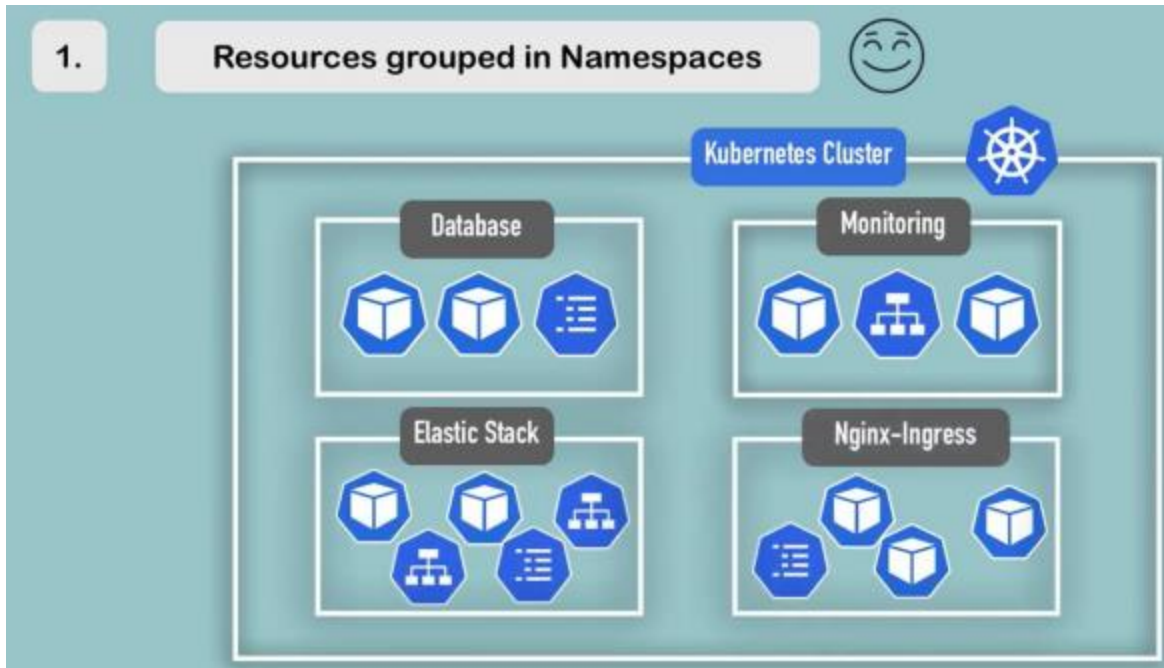
We have physical cluster is available we create different virtual cluster (ns), we may create dev name space for development related applications and prod namespace for production related applications. This way we separate different environments using namespace. If we don't limit the resources dev namespace may use all available resources of the physical cluster.

Why to use Namespace?

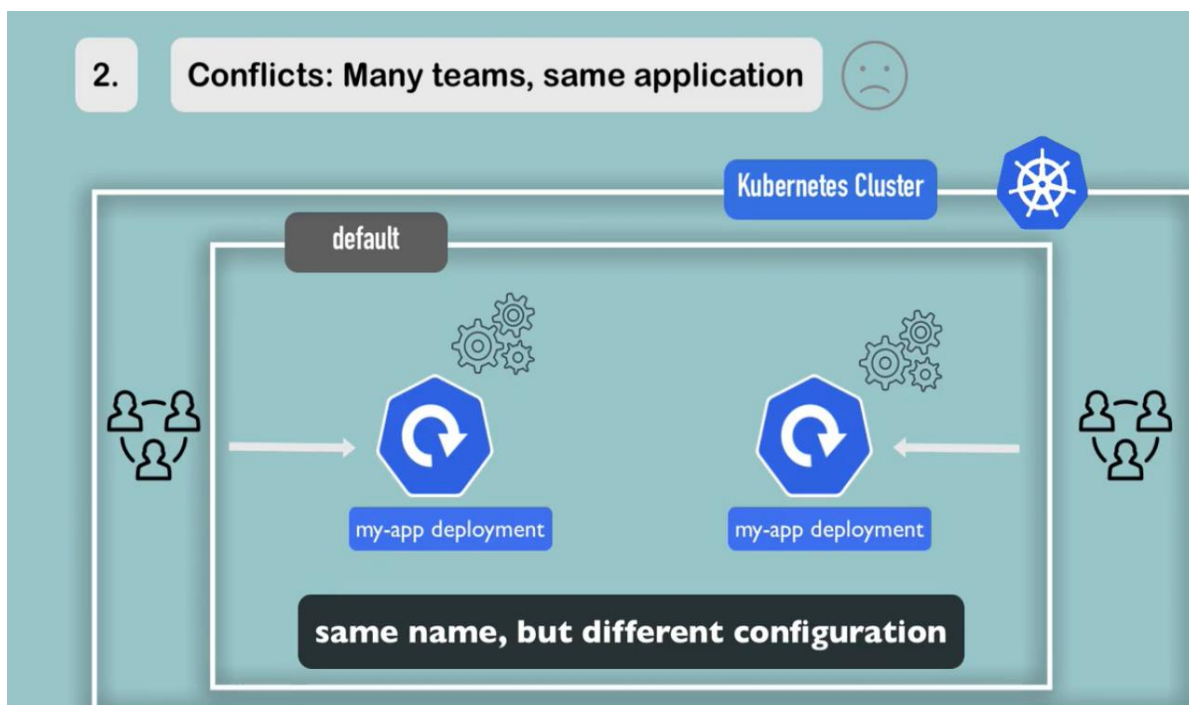
UC1: Everything in 1 namespace?



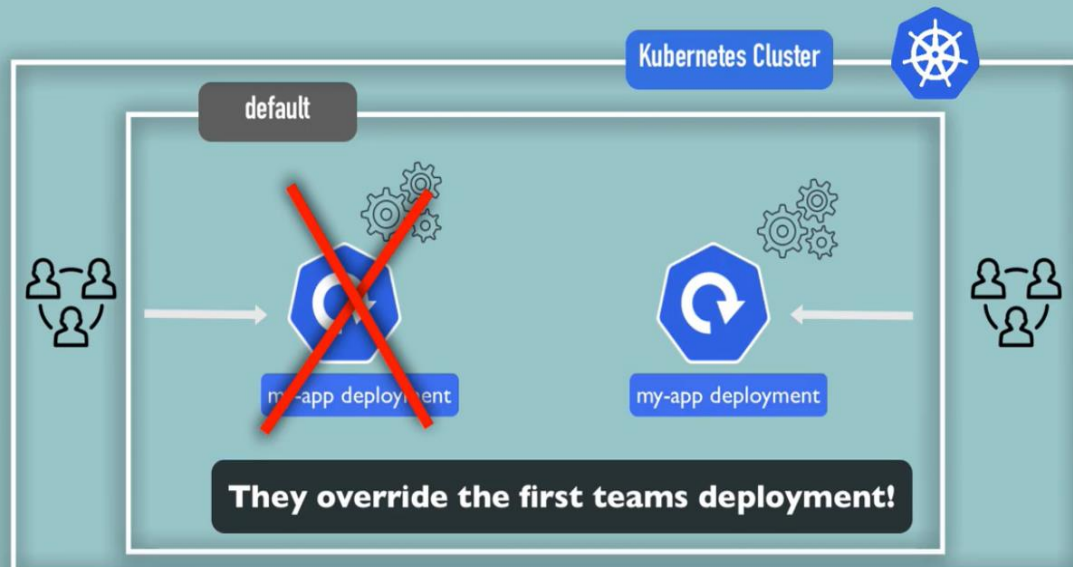
Resources Grouped in Namespaces.



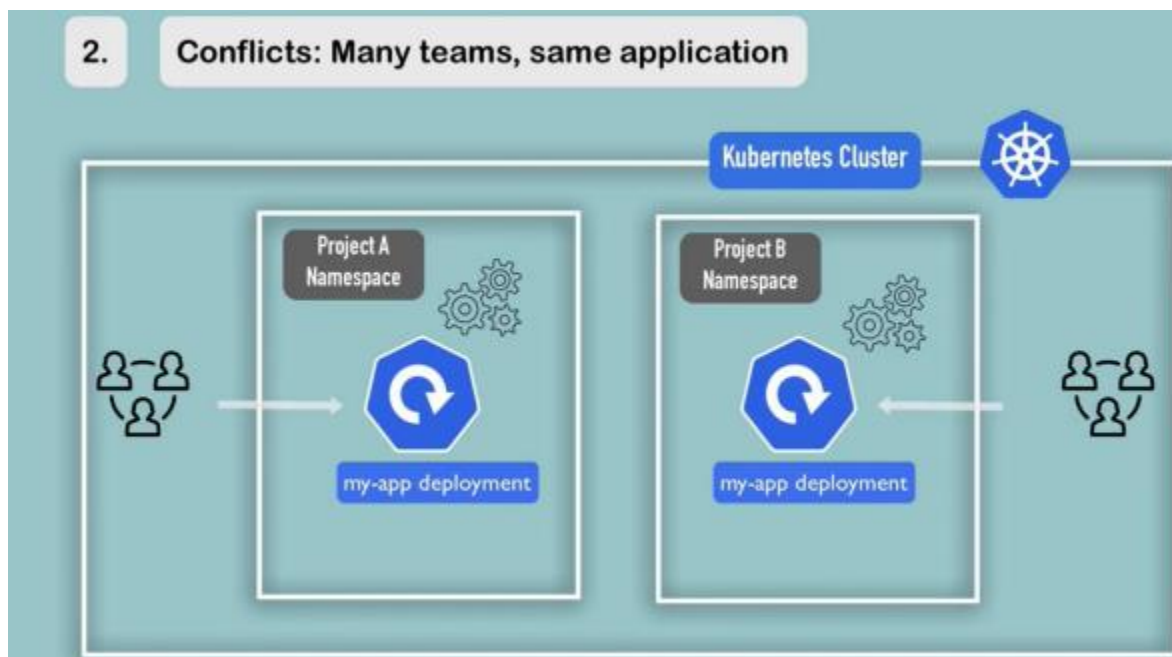
UC2: Conflicts: Many teams, same application.



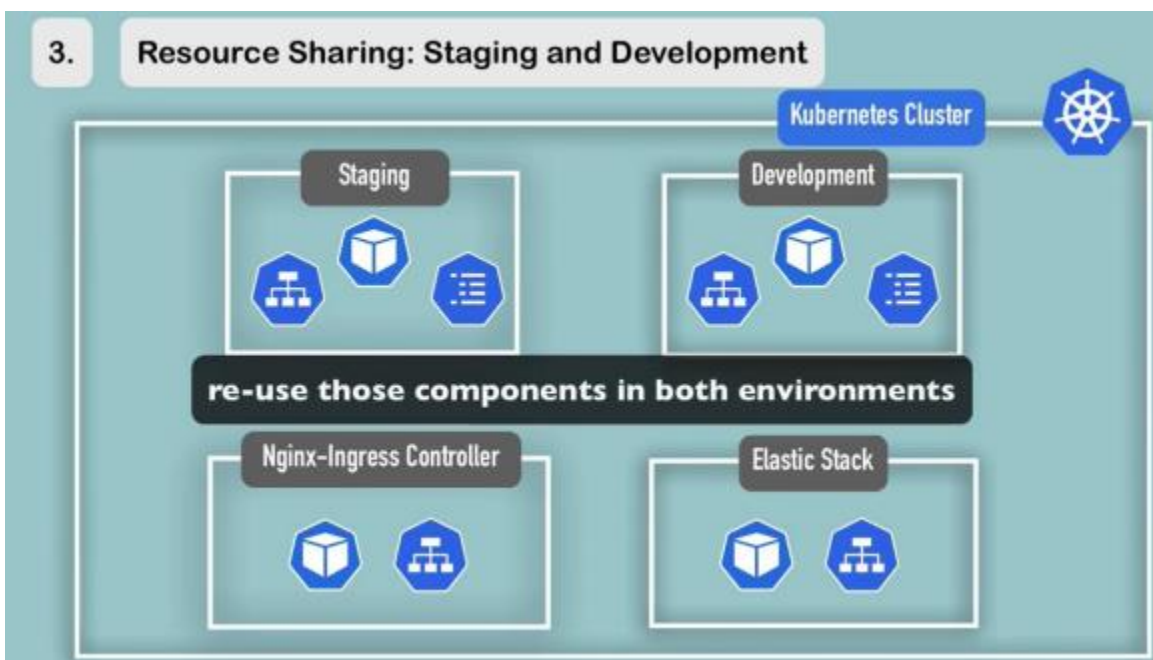
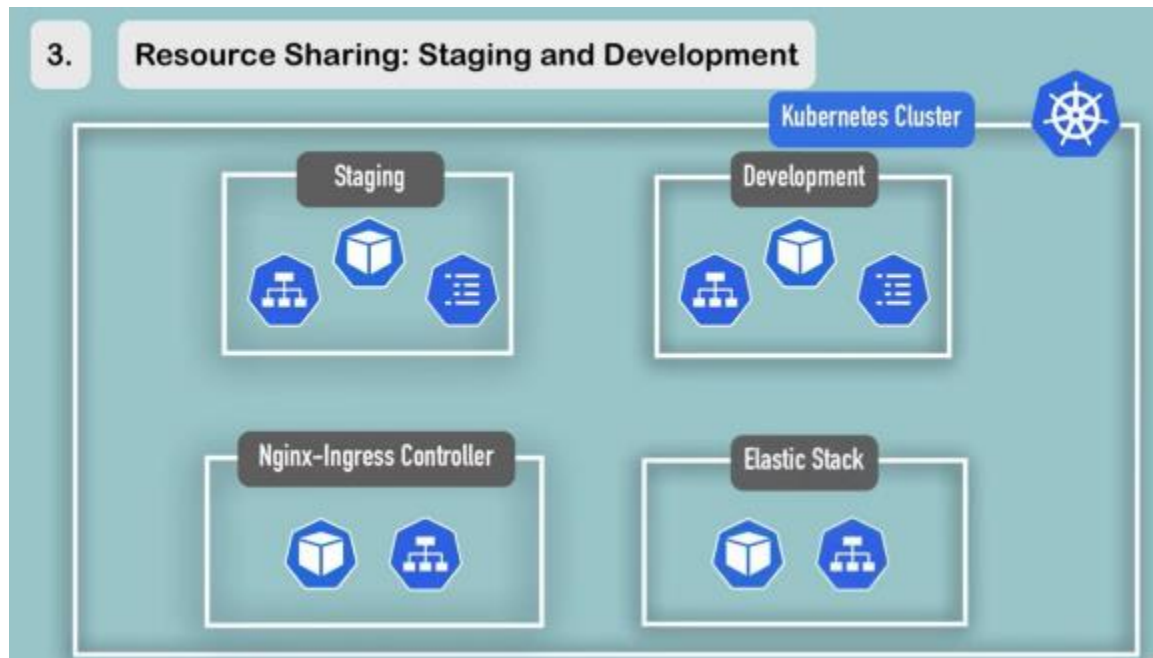
2. Conflicts: Many teams, same application



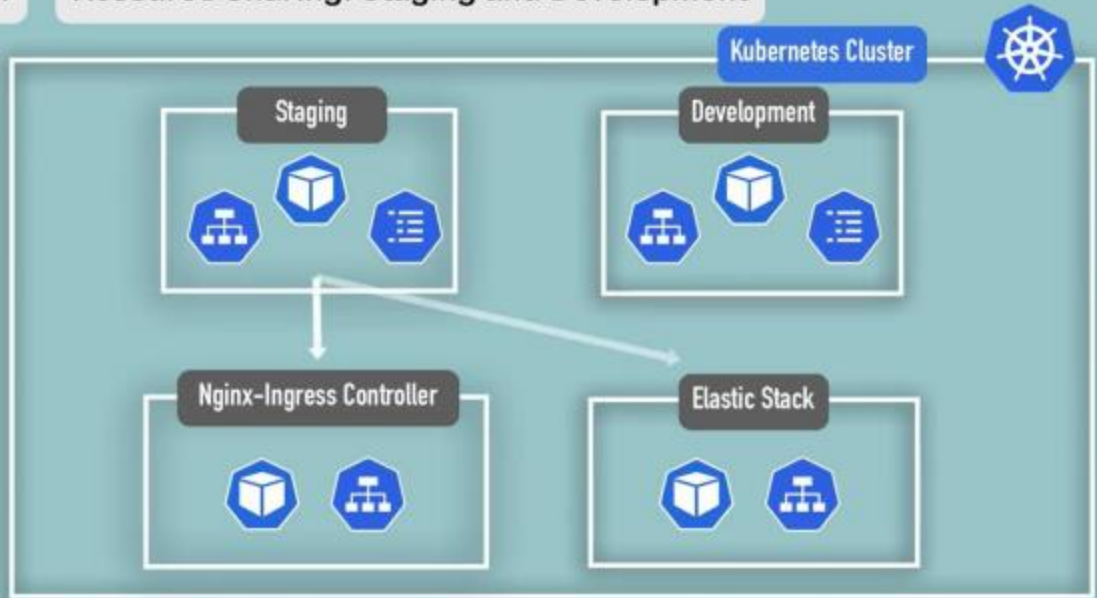
2. Conflicts: Many teams, same application



UC3: Resource Sharing: Staging and Development

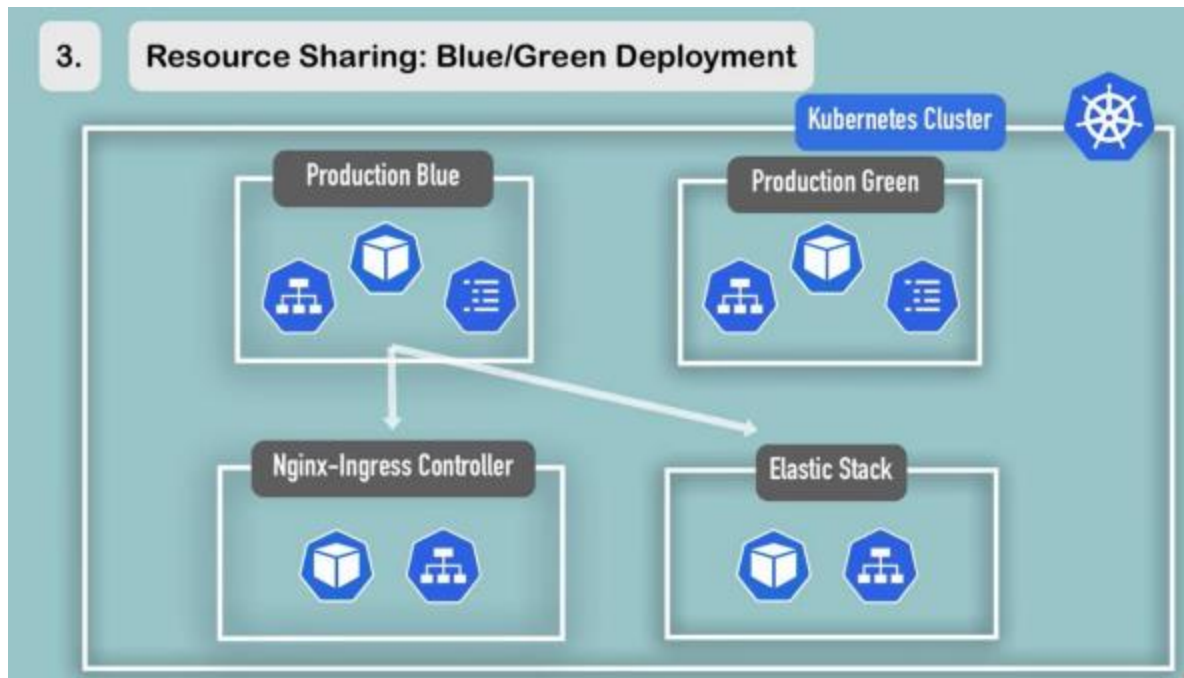


3. Resource Sharing: Staging and Development

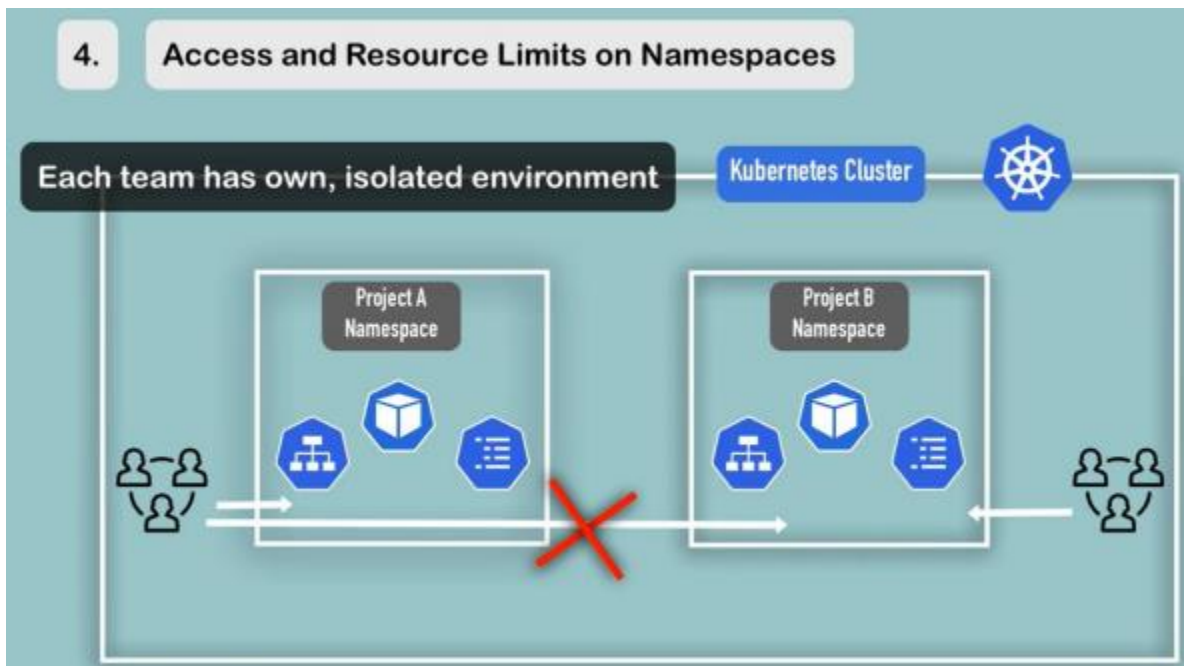


3. Resource Sharing: Blue/Green Deployment

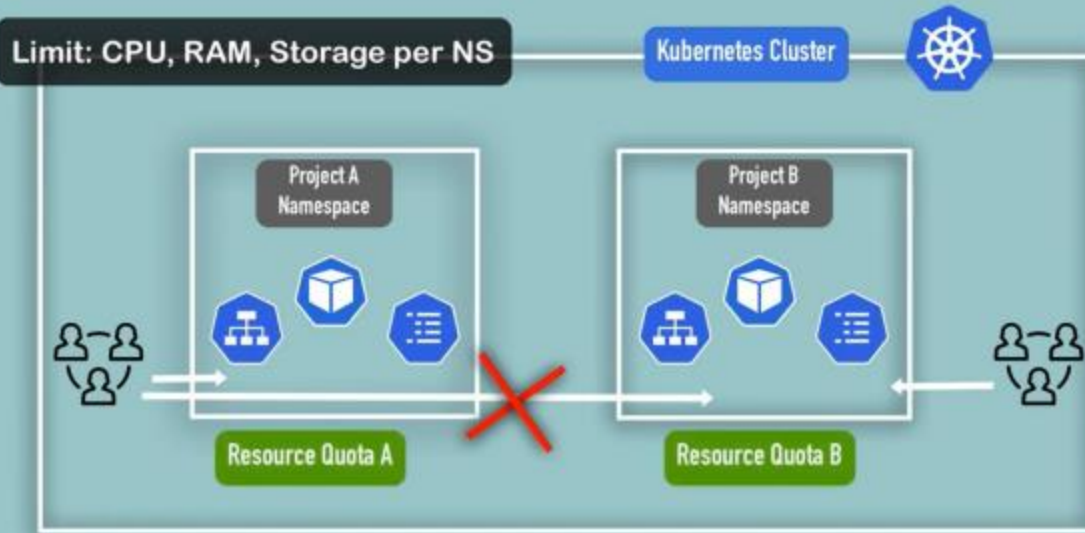




UC4: Access and Resource Limits on Namespace



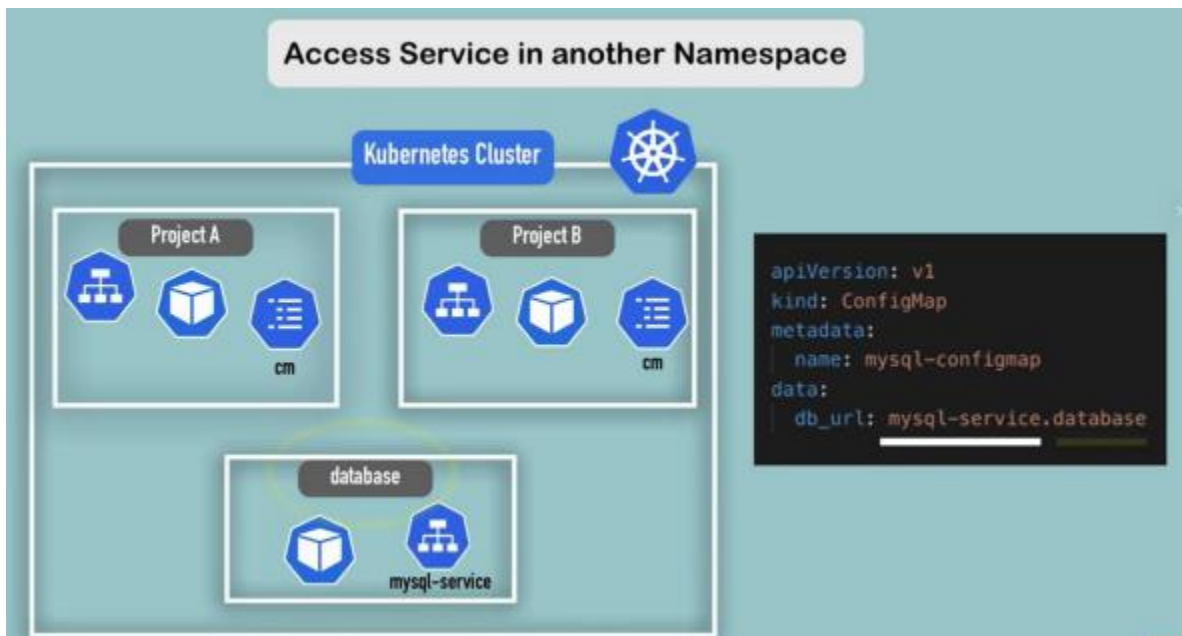
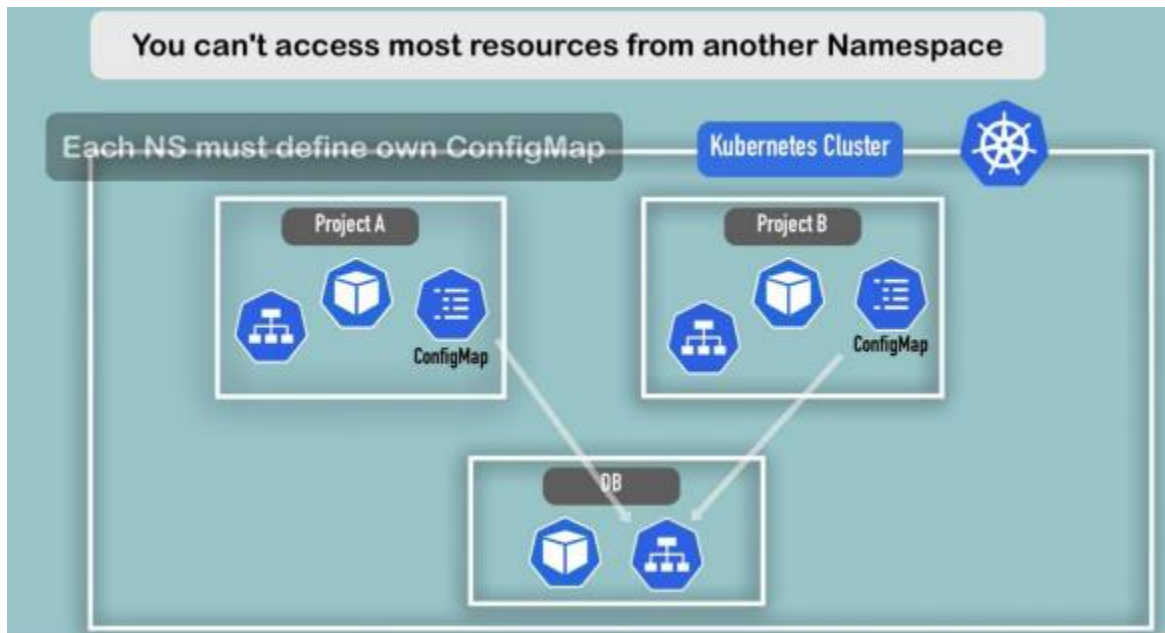
4. Access and Resource Limits on Namespaces

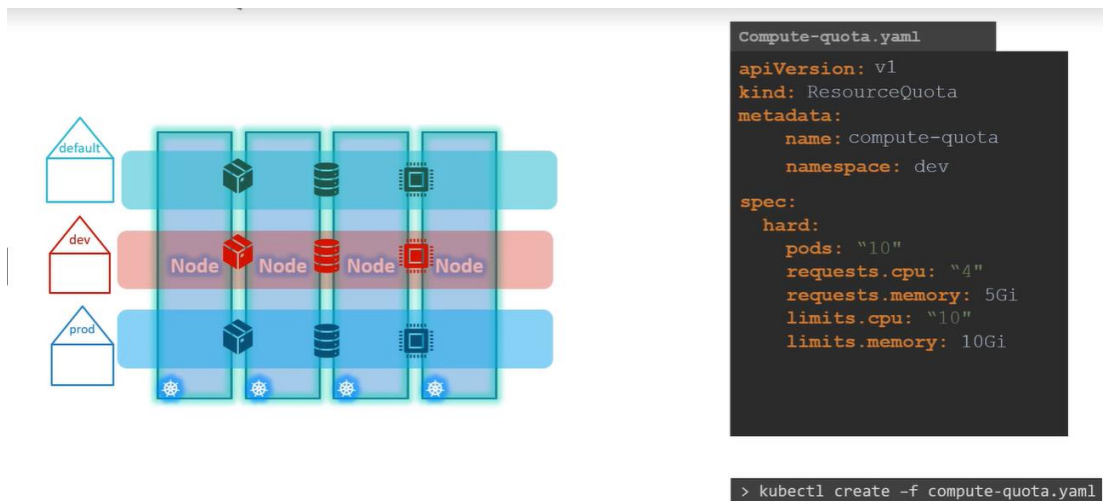


Use Cases when to use Namespaces

1. **Structure** your components
2. **Avoid conflicts** between teams
3. **Share services** between different environments
4. **Access and Resource Limits** on Namespaces Level

Characteristics of the Namespace





A resource quota, defined by a ResourceQuota object, provides constraints that limit aggregate resource consumption per namespace. It can **limit the quantity** of objects that can be created in a namespace by type, as well as the total amount of **compute resources that may be consumed by resources in that project**.

`kubectl describe quota <name> -n ns`

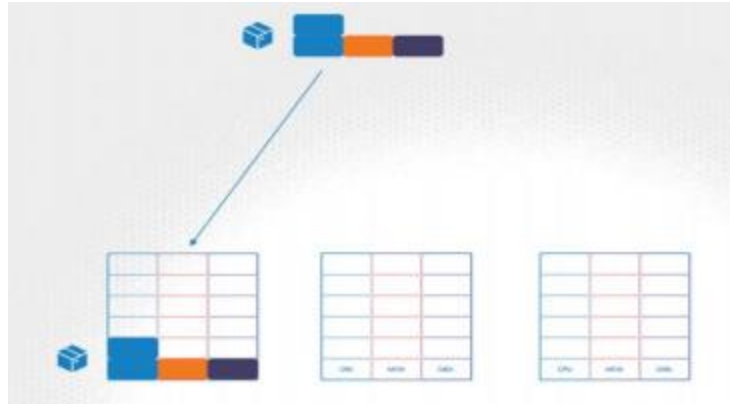
services, deployment is just logical thing don't require any memory, cpu

Note: If quota is already applied on ns, then we can't send object creation request without specify the resource requests. (Show age proof before entering into club)

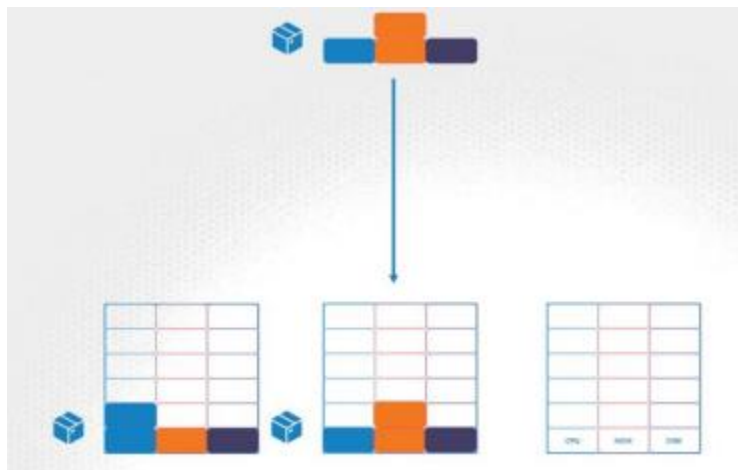
`kubectl run nginx --image=nginx --requests="" --limits="" -n prod`

Resource Requirements

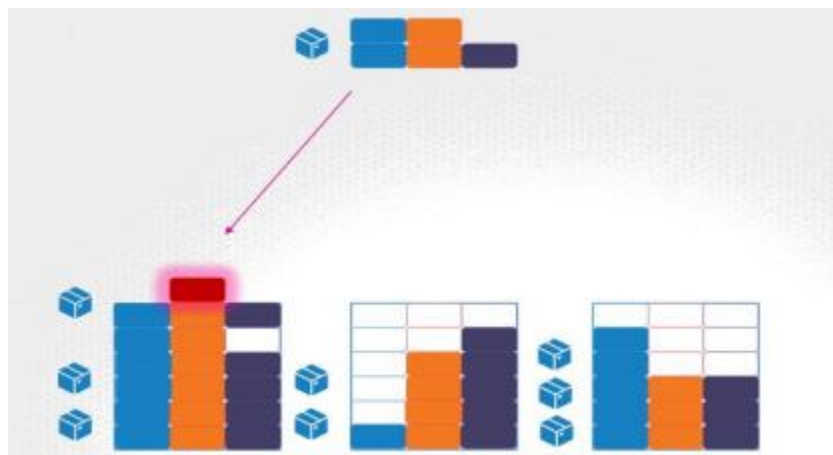
Let us look at a 3 Node Kubernetes cluster. Each node has a set of CPU, Memory and Disk resources available. Every POD consumes a set of resources. In this case 2 CPUs, one Memory and some disk space. Whenever a POD is placed on a Node, it consumes resources available to that node.



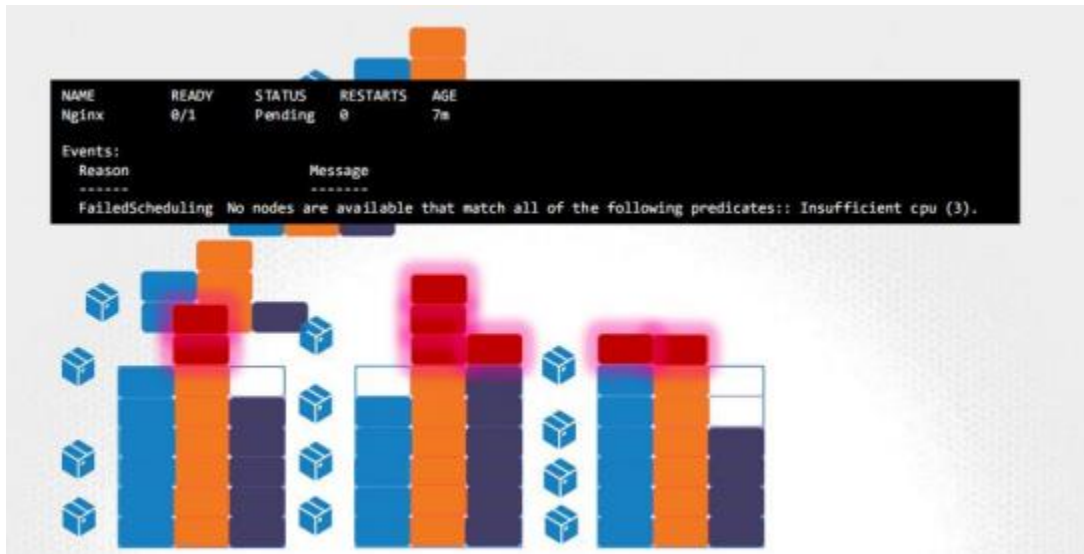
As we have discussed before, it is the kubernetes scheduler that decides which Node a POD goes to. The scheduler takes into consideration, the amount of resources required by a POD and those available on the Nodes. In this case, the scheduler schedules a new POD on Node 2.



If there are no sufficient resources available on any of the nodes, Kubernetes holds back scheduling the POD, and you will see the POD in a pending state.



If you look at the events, you will see the reason – insufficient CPU.

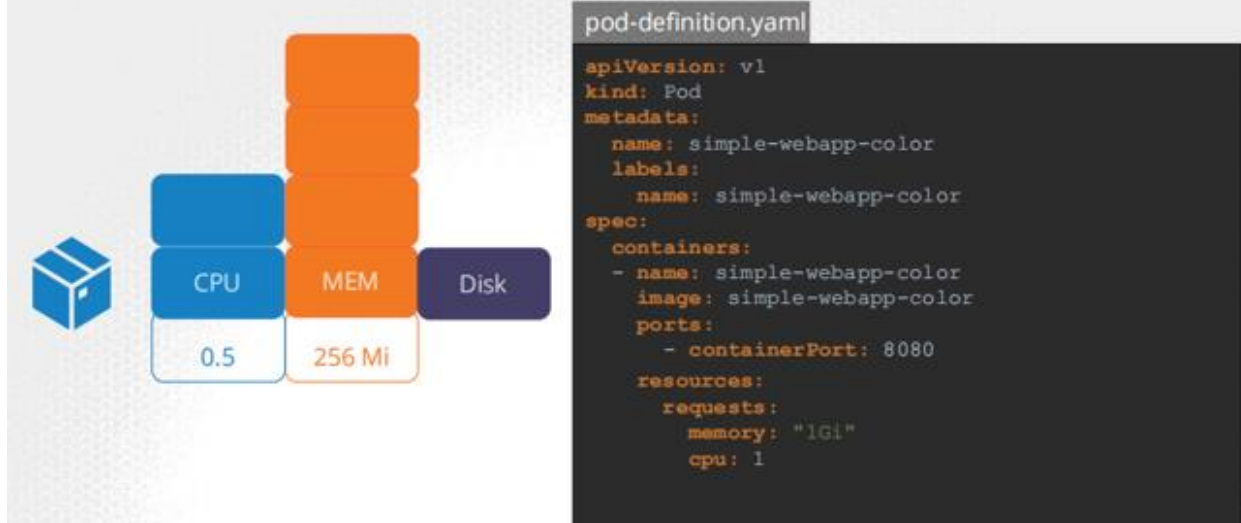


Let us now focus on the resource requirements for each POD. What are these blocks and what are their values? By default, **kubernetes assumes that a POD or a container within a POD requires .5 CPU & 256Mi of memory. This is known as the resource request for a container. The minimum amount of CPU or Memory requested by the container. When the scheduler tries to place the POD on a Node, it uses these numbers to identify a Node which has sufficient amount of resources available.** Now, if you know that your application will need more than these, you can modify these values, by specifying them in your POD or deployment definition files.

In this sample pod definition file, **add a section called resources**, under which add requests and specify the new values for memory and cpu.

UC: In this case I set it to 1GB of memory and 1 count of vCPU.

Resource Requests



Resource Limits

Let's now look at a container running on a Node. In the Docker world, a docker container has no limit to the resources it can consume on a Node. Say a container starts with 1 vCPU on a Node, it can go up and consume as much resource as it requires, suffocating the native processes on the node or other containers of resources. However, you can set a limit for the resource usage on these PODs. By

default Kubernetes sets a limit of 1vCPU to containers. So if you do not specify explicitly, a container will be limited to consume only 1 vCPU from the Node.

The same goes with memory. By default, kubernetes sets a limit of 512 Mebibyte on containers.

If you don't like the default limits, you can change them by adding a limits section under the resources section in your pod definition. Specify new limits for memory and cpu.

Resource Limits



pod-definition.yaml

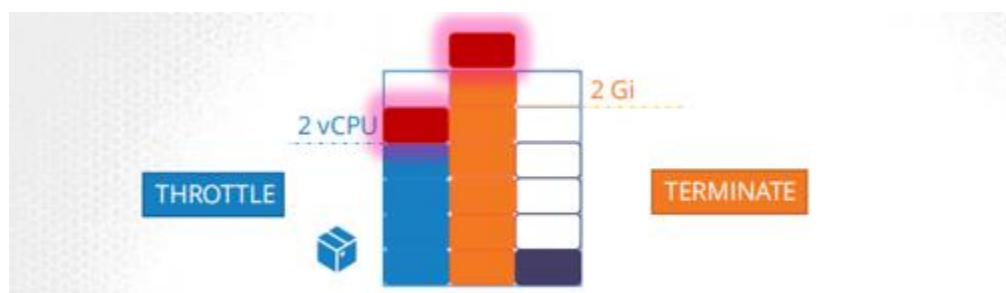
```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    resources:
      requests:
        memory: "1Gi"
        cpu: 1
      limits:
        memory: "2Gi"
        cpu: 2
```

When the pod is created, kubernetes sets new limits for the container. Remember that the limits and requests are set for each container.

So, what happens when a pod tries to exceed resources beyond its specified limit?

In case of the CPU, kubernetes throttles the CPU so that it does not go beyond the specified limit. A container cannot use more CPU resources than its limit.

However, **this is not the case with memory. A container CAN use more memory resources than its limit. So, if a pod tries to consume more memory than its limit constantly, the POD will be terminated.**



Requests and Limits

Request and Limit are the two mechanism that Kubernetes use to manage the CPU and memory resource requirement of the Pods. There are other resources as

well but CPU and memory are the main resources which must be configured properly for containers running in pods.

These resources requirements can be configured at the container level, but because its recommended that one pod should run a single container(although it can run multiple containers), so we will talk in terms of pod resource requirements. Also, if there are multiple containers running inside a pod, the resource requirement of the pod is the combined resource requirements of all the containers running inside it.

Requests is used to specify the amount of resource that should be allocated to any pod for utilization, which means the pod is assigned this much resource when it starts. Whereas the Limit is used to specify the maximum amount of resource available for the pod, it makes sure a pod never goes above a certain value.

When we start a container in a pod using a YAML file with requests and limit value specified, then the Kubernetes scheduler looks for a node which has the required resources for the pod and then the node with enough resources gets allocated to the pod. If no node is found free, then the pod goes into Pending state, and waits for the resources to get free.

If you have automatic scaling enabled in the Kubernetes cluster then a new node is added to the cluster in case of lack of resource. If we do not configure the requests and limits, by default a minimum value is assigned .5 CPU and 256Mi in most Kubernetes services.

Lack of resources can lead to your pods getting evicted frequently due to insufficient memory or CPU. You can use the **kubectl describe** command to find the reason for an evicted pod in your Kubernetes cluster. Here is the command

```
kubectl describe pod POD_NAME -n NAMESPACE_NAME
```